

ALOHAS

Analytics Engineer Study Case

About time.

*ALOHAS makes things designed to last. The numbers
behind the business should hold up just as well.*

The brief

The Data team is looking for an Analytics Engineer to own a corner of how this business is measured — from the dbt layer to the dashboards leadership reads on Monday morning.

This case study is one slice of that work. A small synthetic dataset modeled after how an ALOHAS-shaped business actually flows: sales across multiple channels, returns that happen later, products with real cost economics behind them, and shipments that aren't free.

You'll spend about a week with it. We're not looking for volume. We're looking for the way you think — about grain, about edge cases, about what a report is actually for. The kind of thinking that holds up six months after the slide deck is closed.

What you're working with

You'll get read access to a BigQuery dataset. Three tables, all under:

alohas-recruiting-study-case.production

dim_product

One row per SKU. The product catalog.

COLUMN	TYPE	MEANING
sku	STRING	Primary key. Format <code>SKU-XXXXX</code> .
name	STRING	Product name.
category	STRING	Product category (Dresses, Shoes, Bags, Accessories, Tops, Bottoms, Outerwear, Swimwear).
base_price	NUMERIC	Catalog list price, EUR.
cost	NUMERIC	Unit cost of goods, EUR.

fct_shipment

One row per shipment.

COLUMN	TYPE	MEANING
<code>shipment_id</code>	STRING	Primary key. Format <code>SHP-XXXXXXX</code> .
<code>shipping_method</code>	STRING	<code>standard</code> / <code>express</code> / <code>next_day</code> / <code>economy</code> / <code>pickup</code> .
<code>shipping_cost</code>	NUMERIC	What we paid the carrier, EUR.
<code>country</code>	STRING	ISO-style 2-letter destination code.

`fct_sale_order_line`

One row per sale order line. Partitioned on `created_at`.

COLUMN	TYPE	MEANING
<code>channel</code>	STRING	<code>online</code> / <code>retail</code> / <code>wholesale</code> / <code>marketplace</code> .
<code>sku</code>	STRING	Links to <code>dim_product</code> .
<code>shipment_id</code>	STRING	Links to <code>fct_shipment</code> .
<code>quantity_sold</code>	INT64	Units on the line.
<code>quantity_returned</code>	INT64	Units returned, currently captured on the original row.
<code>gross_sale</code>	NUMERIC	<code>base_price × quantity_sold</code> .
<code>taxes</code>	NUMERIC	Sales tax. Zero for wholesale.
<code>net_sales</code>	NUMERIC	<code>gross_sale – taxes</code> .
<code>created_at</code>	TIMESTAMP	UTC. Two years of history.

A note on the dataset: it's synthetic — generated for this exercise — but the shapes are real. Mixed channels, returns concentrated online, wholesale running on a different tax footing, a Nov/Dec peak, growth on top. Trust the structure, audit the data.

The three things we'd like you to look at

We don't want a kitchen-sink report. We want a focused one. Three questions, in this order.

01 — Channel sales

How is the business performing across channels?

That question is wider than it looks. What's the right time grain? Are we comparing like for like (returns, taxes, channel mix shift)? Is one channel quietly growing while another quietly leaks? When you show channel performance, what would a CEO want to see first — and what would a Head of Wholesale want to see instead?

02 — Net sales and late-arriving returns

In this dataset, `quantity_returned` lives on the same row as the original sale — **and that value is updated in place when the return happens**. A line that reads `quantity_returned = 0` today may read `2` a month from now, without any new row being inserted. The row is mutable.

In reality, returns arrive 30 to 90 days after the transaction. A dashboard built today on this schema would technically work — and then quietly start lying next quarter, as new returns land on old months.

We'd like you to design how this should be modeled. Propose a schema. Decide which metric definitions you'd defend ("net sales as-of date of sale" versus "net sales as-of report date"). Sketch how a chart that looks correct today should behave six months from now when the returns trickle in.

03 — Contribution margin

Each unit sold has costs attached to it. The cost of the product itself (`dim_product.cost`). The cost of getting it to the customer (`fct_shipment.shipping_cost`). And, when applicable, the cost of a return.

Build a view of contribution margin. By channel, by category, by something else if you find something worth showing. Tell us which products and which channels are actually making money — and which ones look healthy on a revenue chart but stop looking healthy here. Be specific about how you handle returns and shipping allocation; the assumptions matter as much as the answer.

What we'd like to receive

A public GitHub repository containing:

- A **report** in whatever format reads best for what you want to say. Interactive HTML is ideal. A static HTML file, a rendered Jupyter notebook, or a PDF are all fine. We'd prefer to read it in a browser than open a slide deck.
- The **code** that produced the numbers — SQL, dbt, Python, whatever you used. Don't polish; let us see the working.
- A **README** with: how to navigate the repo, any assumptions you made, anything you'd do differently with more time.

A few small asks:

- Audit the data before you trust it. If you spot anything that looks wrong, call it out — that's part of the work, not a distraction from it.
- Pick depth over coverage. Three solid answers beat ten thin ones.
- Write like a person, not like a brand. We'd rather read "I assumed X because Y" than "the data demonstrates a robust correlation."

How we'll read your work

We're looking at five things, roughly equally weighted.

WHAT WE LOOK FOR	WHAT "GOOD" LOOKS LIKE
Modeling thinking	You state your grain. You handle returns deliberately. Your proposed schema for late returns could ship.
Reporting craft	Your charts answer specific questions. You write to the reader, not to yourself. The number on every chart is one you'd defend in a meeting.
Code quality	SQL/Python is idiomatic and readable. Logic is layered, not one giant query. Tests, if present, test the right things.
Communication	Assumptions are written down. Tradeoffs are explicit. We can read the report cold and understand the choices.
Awareness	You notice when the data is asking for a second look. You distinguish between modeling problems, data problems, and reporting problems.

There are also two small things that will make us smile if they show up — neither is required, but they are noticed.

- Catching the data quality issues hiding in this dataset. (Yes, there are some.)
- Treating the contribution margin section as a business question, not a math exercise.

Logistics

Time. You have one week from when you receive this brief. We'd rather you spend around **eight hours** total on it than push through twenty. Quality over hours.

Access. Send us the Google email you'd like to use. We'll grant `BigQuery Data Viewer` + `BigQuery Job User` on `alohas-recruiting-study-case` within a working day.

Questions. Reply to the email you got this brief on. Real questions are fine and won't cost you anything in the evaluation. Silent guessing will.

Submission. Send back a link to your GitHub repo with the report rendered inside.

A last word

We're not looking for someone who can produce the most charts.
We're looking for someone who can sit with a question, pick the
angle that matters, and write down a clean answer.

If that's the kind of work you'd enjoy doing every day, we'd love to
see what you come back with.

ALOHAS Data Team